

Software for Multiobjective Optimization in Layered Graphs

Matthias Stallmann

December 16, 2024

Abstract

[\[overview\]](#)

1 Introduction

2 Exact Solutions

Here we discuss mixed integer programming formulations that address three primary objectives and their bottleneck versions. Using these formulations it is possible to optimize one objective subject to restrictions on one or more of the others. Thus it is possible to identify Pareto optima for any combination of objectives.

Section 2.1 describes how a layered graph representation, along with instructions about which objectives are to be optimized or constrained, can be converted to an input file for an ILP solver. Section 2.2 discusses a method for converting the (annotated) ILP solution back to a representation that reflects the optimum layout. Section 2.3 introduces an animation/visualization tool that can be used to display the layouts. A detailed example is presented in Section 2.4.

2.1 Integer/Quadratic Programs to Explore Pareto Optima

The `sgf2ilp.py` program converts a layered graph in `sgf` format to an integer or quadratic program in CPLEX[2] LP format. It supports three different objective functions and their bottleneck versions. Additionally, it also supports optimization of one of the six objective functions given constraints on any of subset of them, including the one being optimized.¹ This leads potentially to $6 \cdot 2^6$ possibilities. In the remainder of this section we present the relevant `ilp` formulations, their complexity in terms of number of variables, constraints, and non-zeroes, as well as experimental results from CPLEX runs.

2.1.1 ILP and Quadratic Programming Formulations

The three primary objectives are crossing minimization and two measures of edge offset, *stretch* and *verticality*. Each of the three is based on a specific measure for an edge.

- *Crossing minimization* counts $c(e)$, the number of times edge e crosses other edges in a given layout. The standard objective is to minimize $(\sum_{e \in E} c(e))/2$, the total number of crossings. The bottleneck version minimizes $\max_{e \in E} c(e)$.
- *Stretch* assumes that nodes on each layer are equidistant and centered on a layer. Suppose the nodes on a layer are permuted using permutation π . If the layer has k nodes, the x-coordinate

¹Most solvers will converge more quickly if they are given a known upper bound.

$x(v)$ of node v is

$$\begin{aligned} &(\pi(x) - 1)/(k - 1) && \text{if } k > 1 \\ &\frac{1}{2} && \text{otherwise} \end{aligned}$$

The x-coordinates are treated as being in the interval $[0, 1]$, so evenly spaced nodes are at coordinates $0, \frac{1}{k-1}, \dots, \frac{k-2}{k-1}, 1$. Given this arrangement, the stretch $s(vw)$ of edge vw is defined as $|x(v) - x(w)|$. Two definitions of total stretch exist: assuming the layers are unit distance apart², we can use the 1-norm to get distance $1 + s(vw)$ for edge vw or the Euclidian distance to get $\sqrt{1 + s(vw)^2}$. We can minimize overall distance if we minimize $\sum_{vw \in E} s(vw)$ or $\sum_{vw \in E} s(vw)^2$, respectively. In the bottleneck version we simply minimize $\max_{vw \in E} s(vw)$ either way.

- *Verticality* assumes nodes are positioned at integer coordinates that need not be contiguous on a layer. If $x(v)$ and $x(w)$ are the coordinates of v and w then $d(v, w) = |x(v) - x(w)|$ is the horizontal displacement of edge vw and $d(v, w)^2$ is the *non-verticality* of vw . Minimization of non-verticality involves solving a quadratic program. However, there is an integer linear program that achieves the same objective, using the observation that $n^2 = \sum_{i=1}^n (2i - 1)$. The bottleneck version minimizes $\max_{vw \in E} d(v, w)$.

For the linear/quadratic programs that can optimize/constrain various combinations of these objectives, we use the following variables, all ≥ 0 . Most come from the ILP formulations in previous work, cited below. Additional variables are introduced later to deal with the multitude of objective combinations.

$x_{i,j}$	=	1	if node i precedes node j on its layer, 0 otherwise
$c_{h,i,j,k}$	=	1	if edge hi crosses edge jk ; 0 otherwise; $h < j$ and $i < k$ to avoid double counting
$p_{i,\ell}$	=	integer	the position, i.e., integer coordinate, of node i on layer ℓ
$d_{u,v,i}$	=	integer	used to linearize $d(uv)^2$
$q_{u,v}$	=	$d(uv)^2$	
$s_{u,v}$	=	$s(uv)$	(floating point) the stretch $s(uv)$ of edge uv
$e_{u,v}$	=	0	dummy variable to recover edges for an sgf representation of the solution

The constraints can be divided into three categories. First the ones related to crossing minimization. Jünger and Mutzel [6] introduced the two-layer versions of these, easily expanded to multiple layers. The addition of constraints for bottleneck crossings is straightforward.

$$x_{i,j} + x_{j,i} = 1 \text{ only one node can precede the other} \quad (1)$$

$$x_{i,k} - x_{i,j} - x_{j,k} \geq -1 \text{ for transitivity; } x_{i,j} = x_{j,k} = 1 \text{ implies } x_{i,k} = 1 \quad (2)$$

$$c_{h,i,j,k} - x_{j,h} - x_{i,k} \geq -1 \text{ condition when edges } hi \text{ and } jk \text{ cross} \quad (3)$$

$$C - \sum_{h,i,j,k} c_{h,i,j,k} \geq 0 \text{ to optimize total crossings or constrain them} \quad (4)$$

$$C_{bn} - \sum_{h,i} c_{h,i,j,k} - \sum_{h,i} c_{j,k,h,i} \geq 0 \text{ for all } jk \in E; \text{ bottleneck crossings} \quad (5)$$

To minimize total or bottleneck crossings, simply minimize C or C_{bn} , respectively. To put upper bound $\tilde{c}$ on total crossings, add constraint $C \leq \tilde{c}$. Upper bounds on any set of objectives can be added similarly.

²It may make more sense to assume the distance between layers is $1/(L - 1)$ if there are L layers; this will not affect what we are minimizing.

Then the ones related to non-verticality, based on Chimani and Hungerländer [1]. Here W is the maximum width, number of nodes, on the widest layer.

$$p_{i,\ell} \leq W - 1 \text{ positions start at 0, cannot be greater than } W - 1 \quad (6)$$

$$p_{i,\ell} - p_{j,\ell} + W + x_{i,j} \geq 1 \text{ ensure } p_{i,j} \text{'s are consistent with } x_{i,j} \text{'s} \quad (7)$$

$$d_{u,v} - p_{u,\ell} - p_{v,\ell+1} = 0 \text{ linear distance between } u \text{ and } v \quad (8)$$

$$d_{u,v,i} - d_{u,v} \leq i \text{ for } 1 \leq i < W; \text{ setup for linearization} \quad (9)$$

$$q_{u,v} - d_{u,v} - \sum_{i=1}^{W-1} 2d_{u,v,i} = 0 \text{ uses fact about sum of odd integers} \quad (10)$$

$$NV - \sum_{uv \in E} q_{u,v} = 0 \text{ total non-verticality} \quad (11)$$

$$NV_{bn} - q_{u,v} \geq 0 \text{ for all } uv \in E; \text{ bottleneck non-verticality} \quad (12)$$

For non-verticality it is also useful to add lower bounds. If a node u has k neighbors on the layer above (or below) it, the non-verticalities of the corresponding edges must range from 1 to $\lfloor k/2 \rfloor^2$ on one side and 1 to $\lfloor (k-1)/2 \rfloor^2$ on the other. This leads to

$$\sum_v q_{u,v} \geq \sum_{i=1}^{\lfloor (k-1)/2 \rfloor} i^2 + \sum_{i=1}^{\lfloor k/2 \rfloor} i^2 \quad \text{for any } u \text{ that has outdegree } k \quad (13)$$

$$\sum_u q_{u,v} \geq \sum_{i=1}^{\lfloor (k-1)/2 \rfloor} i^2 + \sum_{i=1}^{\lfloor k/2 \rfloor} i^2 \quad \text{for any } v \text{ that has indegree } k \quad (14)$$

$$NV_{bn} \geq \lfloor k/2 \rfloor \text{ where } k \text{ is the maximum indegree or outdegree of any node} \quad (15)$$

The lower bounds (13) and (14) are effective even if NV_{bn} is the only relevant objective, but in most cases only if $k > 3$.

Finally the ones related to stretch – see the 2016 technical report [10]. These also include (6) and (7), but, instead of the maximum width W , we need to consider the width of each separate layer. The current implementation does not allow for combinations of objectives that involve both non-verticality and stretch. We introduce $z_{i,j}$ for any pair i, j of nodes on adjacent layers and let the constant $W[i]$ represent the width of the layer containing node i .

$$z_{u,v} + \frac{1}{W[i]} p_i - \frac{1}{W[j]} p_j = 0 \text{ actual difference in centered positions of } i \text{ and } j \quad (16)$$

$$s_{u,v} + z_{u,v} \geq 0 \text{ ensure } s_{u,v} \geq |z_{u,v}| \text{ if } z_{u,v} \text{ is negative} \quad (17)$$

$$s_{u,v} - z_{u,v} \geq 0 \text{ ensure } s_{u,v} \geq |z_{u,v}| \text{ if } z_{u,v} \text{ is positive} \quad (18)$$

$$S_{bn} - s_{u,v} \geq 0 \text{ where } S_{bn} \text{ is bottleneck stretch} \quad (19)$$

$$S - \sum_{u,v} s_{u,v} \geq 0 \text{ if a bound } S \text{ on total stretch is specified} \quad (20)$$

To obtain a desired combination of an objective with a set (possibly empty) of bounds on objectives, `sgf2ilp.py` has options described in Fig. 1.

Only those variables and constraints that are required for the desired objectives are included. However, the $p_{i,\ell}$ and $e_{u,v}$ variables must be included so that an `sgf` file for the solution can be constructed.

2.1.2 Sizes of the Integer/Quadratic Programs

In the discussion that follows we introduce relevant notation. Let n_ℓ be the number of nodes on layer ℓ and $n_{\max}$ be the maximum number of nodes on any layer. The *channel* ℓ , below layer ℓ , is

```

--objective {total,bottleneck,vertical,bn_vertical,quad_vertical,
            stretch,bn_stretch,quad_stretch}
    minimize ...
        total/bottleneck (total/bottleneck crossings)
        vertical/bn_vertical (minimize total/bottleneck non-verticality)
        quad_vertical (use quadratic programming to minimize non-verticality)
        stretch/bn_stretch (total/bottleneck edge length with evenly spaced nodes)
        quad_stretch (use quadratic programming to minimize total stretch
                        with the stretch of each edge squared)
--total TOTAL
    constraint on the total number of crossings (default: None)
--bottleneck BOTTLENECK
    constraint on the maximum number of crossings of any edge (default: None)
--stretch STRETCH
    constraint on the total edge length (default: None)
--bn_stretch BN_STRETCH
    constraint on the maximum length of any edge (default: None)
--vertical VERTICAL
    constraint on the total non-verticality (default: None)
--bn_vertical BN_VERTICAL
    constraint on the maximum non-verticality of any edge (default: None)

```

Figure 1: Options for the `sgf2ilp.py` program.

the set of edges between nodes on layer ℓ and nodes on layer $\ell - 1$. Define m_ℓ to be the number of edges in channel ℓ and $m_{\max}$ to be the maximum number of edges in any channel.

The number of variables for each objective is bounded as follows.

- the position variables $p_{i,\ell}$ and the related $x_{i,j}$ are required for all objectives; there are n of the former and $\Theta(n_\ell^2)$ of the latter on layer ℓ . Since $\sum_\ell n_\ell^2$ is $O(n_{\max}^2)$, the total is $O(n_{\max}^2)$; the worst case is when there is one large layer, so effectively $O(n^2)$.
- the number of crossing variables $c_{h,i,j,k}$ for channel ℓ is $\Theta(m_\ell^2)$ and the sum of these is $O(m_{\max}^2)$; the worst case arises when there is one large channel, so effectively $O(m^2)$.
- the number of variables related to verticality is dominated by the $d_{u,v,i}$'s, so $mn_{\max}$; usually, $m \geq n$, so this is the dominant term.
- there are exactly m stretch variables $s_{u,v}$; the number of position variables is $O(n^2)$ and m is $O(n^2)$ as well.

The quadratic programming versions of verticality and stretch have no more variables than the linear ones. In the case of verticality, there are no $d_{u,v,i}$ variables so the total is m . The relevant numbers are summarized in Table 1(a).

The numbers of constraints for each objective are as follows with a summary in Table 1(b).

- for crossings, the dominant constraints are related to transitivity and edge crossings; in the former, every triple of nodes on each layer is involved, so we have $\sum_\ell n_\ell^3$ with upper bound $O(n^3)$; in the latter, every pair of edges in each channel is involved, so $\sum_\ell m_\ell^2$, which is $O(m^2)$.
- for verticality, the constraints that set values for the $d_{u,v,i}$'s, constraints (9) dominate, so we have $O(mn)$; if quadratic programming is used, only (6), (7), and (8) are needed, so we have exactly n for (6), $\sum_\ell n_\ell^2$ for (7), and m for (8); this is $O(n^2)$.
- for stretch, sets (16), (17), and (18) each have one constraint per edge, so the total is $O(m)$; but we also have to have position constraints (6) and (7), so the total is $O(n^2)$ based on (7); only the objective function differs in the quadratic version.

objective	detailed	upper bound
crossings	$\sum_{\ell} m_{\ell}^2$	$O(m^2)$
verticality	$mn_{\max}$	$O(mn)$
quadr. verticality	m	$O(m)$
stretch/position	$\sum_{\ell} n_{\ell}^2$	$O(n^2)$

(a) number of variables

objective	upper bound	dominant constrs.
crossings	$O(m^2 + n^3)$	(2) and (3)
verticality	$O(mn)$	(9)
quadr. verticality	$O(n^2)$	(7)
stretch/position	$O(n^2)$	(7)

(b) number of constraints

objective	upper bound	dominant constraints
crossings	$O(m^2 + n^3)$	(2), (3) (4), (5)
verticality	$O(mn)$	(9) and (10)
quadr. verticality	$O(n^2)$	(7)
stretch/position	$O(n^2)$	(7)

(c) number of non-zeros

Table 1: Sizes of integer/quadratic programs.

The input file size is determined by the total size of all the constraints, i.e., the number of nonzeros in the constraint matrix. These are detailed below with a summary in Table 1(c).

- crossings: constraints (1), (2), and (3) are constant size and there are $O(m^2 + n^3)$ of them; constraints (4) and (5) are size $O(m^2)$ each and there are two of them, so the total is $O(m^2 + n^3)$.
- verticality: only two constraints have non-constant size; they are the odd integer summations in (10) and the total non-verticality constraint (11); of these the summations clearly dominate; there are m constraints, each of size up to n , so $O(mn)$; these constraints are not needed for quadratic programming, but the number of position consistency constraints (7) dominates.
- stretch: the only constraint of non-constant size is (20) and it has size $m+1$; again, the position constraints dominate.

So the number of nonzeros, while being larger than the number of constraints, is asymptotically the same. The only ones of non-constant size have size equal to the number of some of the constant size constraints.

2.1.3 Experimental Results

Experiments were done using CPLEX version 12.6 on an Intel(R) Xeon(R) Platinum (multi-)processor running Centos 7 Linux. There are 32 2.1 GHz processors, each with 22 KB cache, and a total of 128 GB shared memory. CPLEX takes full advantage of multiprocessing. In addition to the runtime reported by `cplex.ilp`, CPLEX reports the total time taken on all processors, which sometimes an order of magnitude greater.

[scatterplots showing nodes/layers and nodes/max edges per channel for north, rome, other, random]
[include experimental results – runtimes for each objective as a function of number of edges or max edges in channel.]

	00-Instance c6-total	c created 2023/07/06 ...
	...	c from a cplex_ilp solution
	runtime 0.027115	c runtime 0.027115
	...	c TimedOut 0
c a cycle with 6 nodes,	BeginSolution	c SolutionFound 1
c and 2 layers	total 2	c ProvedOptimal 1
t c6 6 6 2	...	c value 2
n 0 0 0	p_0_0 0	t c6-total 6 6 2
n 1 0 1	p_1_0 2	n 0 0 0
n 2 0 2	p_2_0 1	n 1 0 2
n 3 1 0	p_3_1 1	n 2 0 1
n 4 1 1	p_4_1 2	n 3 1 1
n 5 1 2	p_5_1 0	n 4 1 2
e 0 3	e_0_3 0	n 5 1 0
e 0 5	e_0_5 0	e 0 3
e 1 3	e_1_3 0	e 0 5
e 1 4	e_1_4 0	e 1 3
e 2 4	e_2_4 0	e 1 4
e 2 5	e_2_5 0	e 2 4
	...	e 2 5
	EndSolution	

(a) original sgf file

(b) cplex_ilp output

(c) sol2sgf.py output

Figure 2: Stages in creating an sgf file from a cplex_ilp solution.

[scatterplots with one of the measures on the x-axis and the other on the y-axis; one series for each objective]

[report number of nonzeros in relation to max nodes per layer and max edges per channel]

[report runtimes as a function of number of nonzeros and number of edges per channel; see which measures give more interesting scatterplots]

2.2 From Solutions to Layouts

The next step is to run `cplex_ilp` on files generated by `sgf2ilp.py`. The incantation

```
cplex_ilp -solution -time=LIMIT FILE.lp > FILE.out
```

will create output with a variety of statistics from the CPLEX run and, more importantly, the values of all variables in the optimum solution found, or best feasible in case of a timeout. The script `sol2sgf.py`, a filter from standard input to standard output, can be used to generate an sgf file with the node positions corresponding to the optimum solution. Presence of the dummy $e_{u,v}$ variables means that the original sgf file need not be a separate input. The sgf file produced by `sol2sgf.py` includes important information such as runtime and the value of the objective function as comments.

Fig. 2 shows relevant parts of the process. Fig. 2 (b) shows only the useful parts of the solution. Values of all other variables are listed in the output as well, preceded by various statistics from the run. The ILP generated by `sgf2ilp.py` in this case had total crossings as the objective, as evidenced by the value of variable `total`. Fig. 3 shows the drawings based on the original sgf file and the one based on the results of the pipeline. The next section discusses the generation of the drawings.

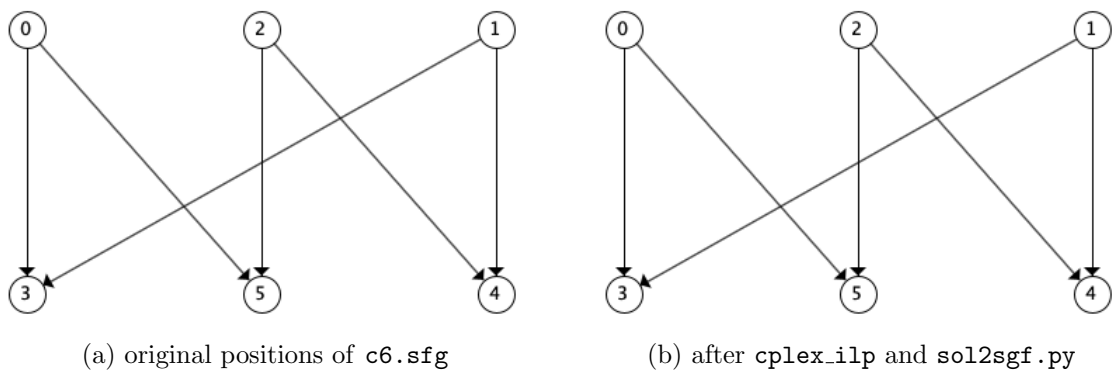


Figure 3: Drawings of a six-node cycle, before and after crossing minimization.

2.3 Visualization using Galant

Galant [4] is an algorithm animation tool that can also be used to create drawings of graphs, both general graphs and layered. The `sgf2layeredgraphml.py` script in the `scripts` directory of the repository creates graphml files designed specifically for display and animation of layered graphs using Galant – so they may not be useable in other contexts.³ Like the scripts described earlier it reads from standard input and writes to standard output. The Galant repository also includes layered graph algorithms.

Fig. 4 illustrates a Galant animation of the barycenter algorithm. A user can step forward or backward in the sequence of displayed steps using the right and left arrow keys, respectively. Holding down the keys will create a movie-like effect. A banner at the top of the window, not shown, displays the current and minimum number of total and bottleneck crossings. Galant animations of barycenter, modified barycenter⁴, sifting [7], and mce [9] are in the `Research/Layered-Graphs/Algorithms` directory of the repository. The `parallel Graphs` directory has example graphs. The animations are easy to develop; they are essentially Java programs with a straightforward API that accesses both standard layered graph operations, such as assigning weights and sorting, along with simple animation effects.

2.4 A Case Study in Seven Drawings

Here we illustrate how our software is used to create eight different drawings of the same graph, `North42.32`, one of over a thousand graphs collected by Stephen North, as layered using an algorithm of Gansner et al.[5]. The first set of four illustrates tradeoffs between total and bottleneck crossings; the second set show tradeoffs between crossings and verticality. In both cases the script `sgf2opt.sh` identifies the extreme points of the Pareto frontier and `pareto_frontier.sh` finds the remaining points.

2.4.1 Total Crossings versus Bottleneck Crossings

First a run of `sgf2opt.sh`.⁵ The `North42.32` instance is in file `north42.32.sgf`.

```
sgf2opt.sh north42.32.sgf total bottleneck 7200
```

The last numerical argument is optional; it specifies a time limit for the CPLEX runs in seconds – default is one hour: here we allow two hours. In the descriptions below, assume that `cplex_ilp` has an additional flag `-time=7200`. The script creates four integer programs and solves them using `cplex_ilp`, a frontend for CPLEX.⁶ The first two runs establish the minimum number of total and bottleneck crossings, respectively, using the `sgf2ilp.py` script to generate the ILP's, and creating output files from the CPLEX runs.

```
sgf2ilp.py --objective total < north42.32.sgf > north42.32-total.lp
cplex_ilp -solution north42.32-total.lp > north42.32-total.out
sgf2ilp.py --objective bottleneck < north42.32.sgf > north42.32-bottleneck.lp
cplex_ilp -solution north42.32-bottleneck.lp > north42.32-bottleneck.out
```

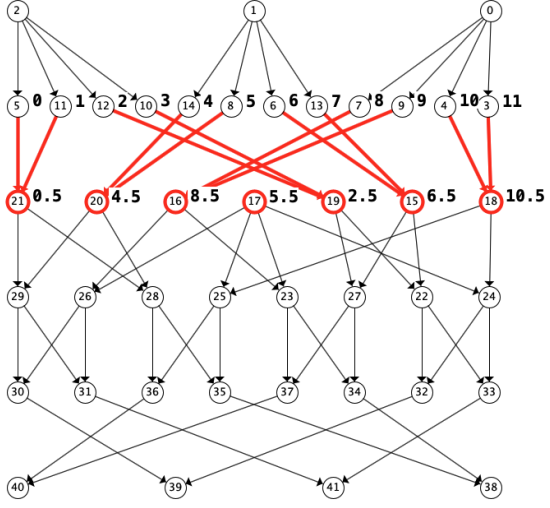
The `-solution` flag tells `cplex_ilp` to output values of variables in a format that allows later conversion to an sgf file. The script harvests the objective values from these two runs: 44 for minimum total crossings and 3 for minimum bottleneck crossings. Then it runs

³Attributes for nodes include `layer` and `positionInLayer` in place of the traditional x- and y-coordinates.

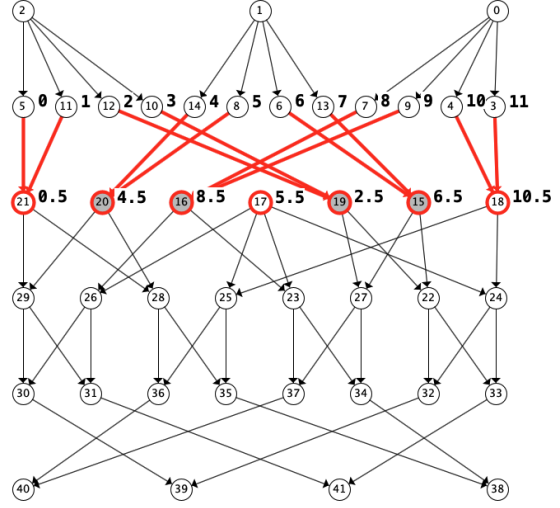
⁴Each pass (i) identifies the layer ℓ whose incident edges have the most crossings; (ii) sorts ℓ based on positions of neighbors on layers both above and below; (iii) does an upward sweep starting with layer $\ell + 1$ and a downward one starting with $\ell - 1$.

⁵It is best to run the scripts within a directory containing the sgf input, being aware that multiple files will be created in the process. Each script invocation needs to be preceded by the path to `repository/ilp`.

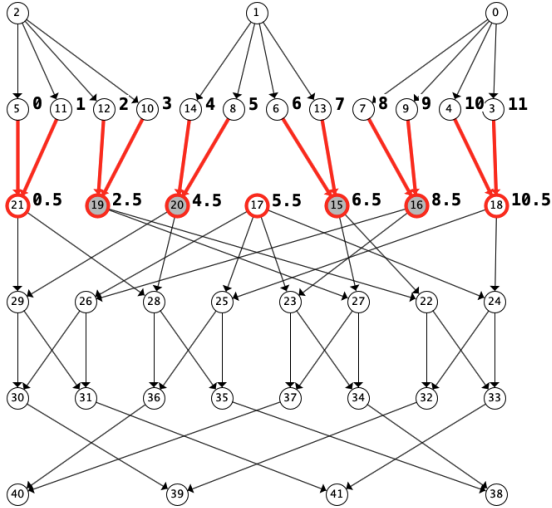
⁶Available at <https://github.com/mfms-ncsu/CPX-ILP>.



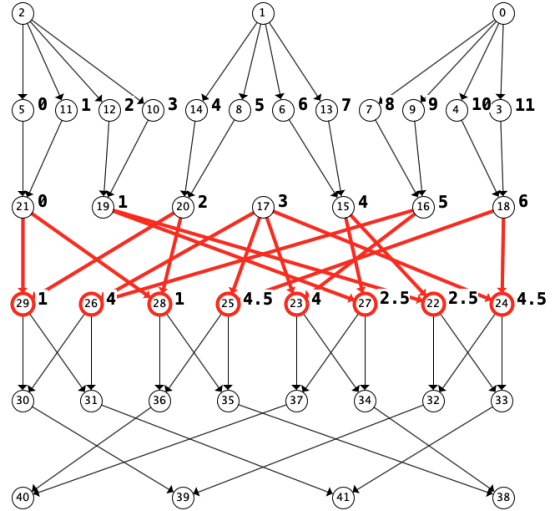
(a) assigning weights to nodes on a layer



(b) marking nodes that will change positions



(c) after sorting by weight



(d) assigning weights on next, lower, layer

Figure 4: The Galant animation of the barycenter algorithm in action.

```
sgf2ilp.py --objective total --bottleneck 3 < north42.32.sgf > north42.32-bottleneck_3_total.lp
cplex_ilp -solution north42.32-bottleneck_3_total.lp > north42.32-bottleneck_3_total.out
sgf2ilp.py --objective bottleneck --total 44 < north42.32.sgf > north42.32-total_44_bottleneck.lp
cplex_ilp -solution north42.32-total_44_bottleneck.lp > north42.32-total_44_bottleneck.out
```

The output summarizes the outcome, showing that, when bottleneck crossings are bounded by 3, total crossings have minimum 46; and when total crossings are bounded by 44, bottleneck crossings have minimum 7. We can then do another run to establish an intermediate Pareto point if one exists.

```
sgf2ilp.py --objective bottleneck --total 45 < north42.32.sgf > north42.32-total_45_bottleneck.lp
cplex_ilp -solution north42.32-total_45_bottleneck.lp > north42.32-total_45_bottleneck.out
```

When total crossings are bounded by 45, bottleneck crossings have minimum 5.

Fig. 5 illustrates these Pareto optima. To generate these pictures, we first do

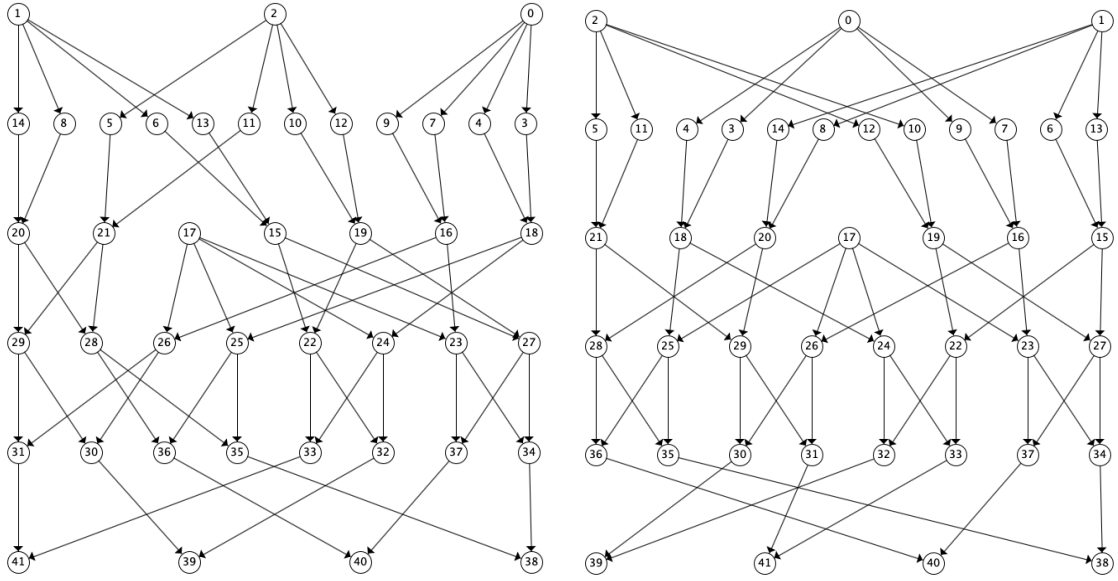
```
sol2sgf.py < north42.32-total_44_bottleneck.out > north42.32-total_44_bottleneck.sgf
sol2sgf.py < north42.32-total_45_bottleneck.out > north42.32-total_45_bottleneck.sgf
sol2sgf.py < north42.32-bottleneck_3_total.out > north42.32-total_46_bottleneck.sgf
```

to create sgf files from the solutions embedded in the `cplex_ilp` output.

Finally, the `scripts` directory in the repository has `sgf2layeredgraphml.py`, used to create inputs for Galant.

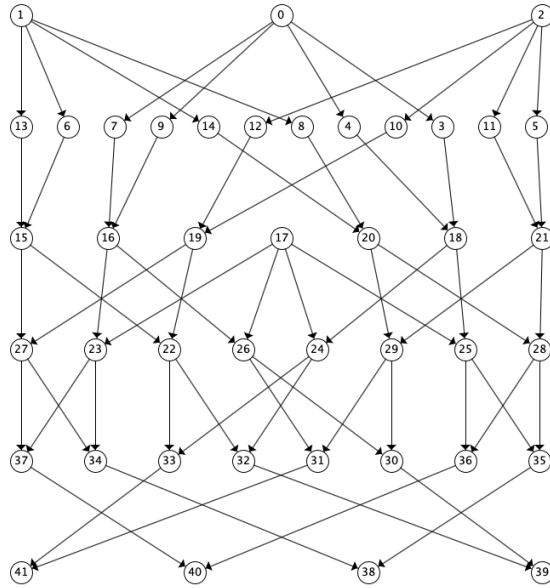
```
sgf2layeredgraphml.py < north42.32-total_44_bottleneck.sgf > north42.32-total_44_bottleneck.graphml
sgf2layeredgraphml.py < north42.32-total_45_bottleneck.sgf > north42.32-total_45_bottleneck.graphml
sgf2layeredgraphml.py < north42.32-total_46_bottleneck.sgf > north42.32-total_46_bottleneck.graphml
```

Of course, the names can be shortened along the way and the suffixes can include information about both objectives, as in `t44b7`, `t45b5`, and `t46b3`.



(a) 44 crossings, 7 bottleneck crossings

(b) 45 crossings, 5 bottleneck crossings



(c) 46 crossings, 3 bottleneck crossings

Figure 5: Three drawings of `north42.32_GKNV`, showing all possible tradeoffs between total crossings and bottleneck crossings.

2.4.2 Total Crossings versus Non-Verticality

To explore tradeoffs between total crossings and non-verticality, we begin with

```
sgf2opt.sh north42.32.sgf total vertical [runtime]
```

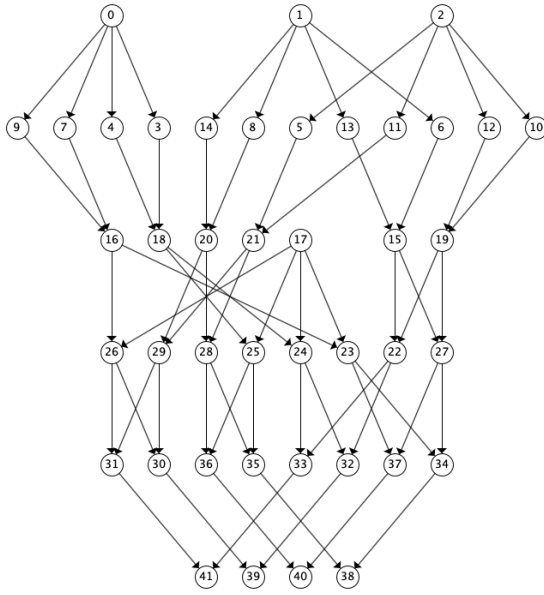
Results give the two extreme points: with minimum total crossings, 44, the minimum achievable non-verticality is 172; with minimum non-verticality, 150, total crossings are minimized at 48. This leaves three potential intermediate Pareto points. We can use the script `pareto_frontier.sh` to explore these as follows.

```
pareto_frontier vertical total 45 47 [runtime]
```

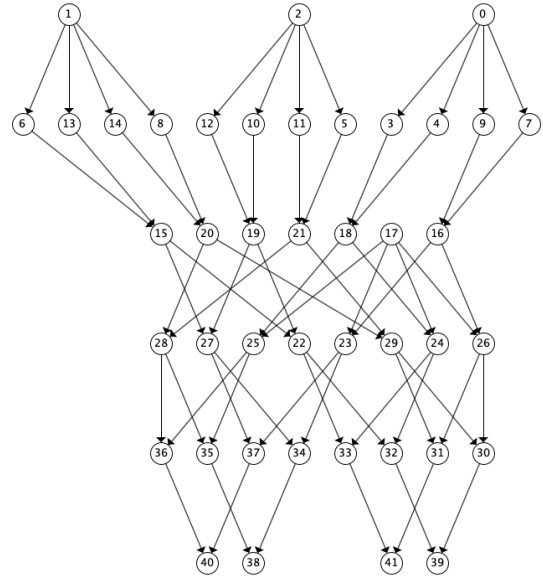
which runs

```
sgf2ilp.py --objective bottleneck --total N < north42.32.sgf > north42.32-total_N_bottleneck.lp  
cplex_ilp -solution north42.32-total_N_bottleneck.lp > north42.32-total_N_bottleneck.out
```

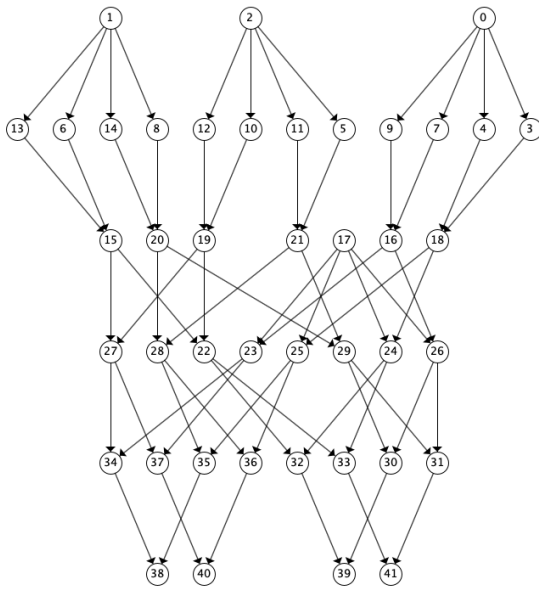
for $N = 45\text{--}47$. The results, along with the extreme points, give us the following (tc, nv) pairs, where tc is total crossings and nv is (total) non-verticality: $(44, 172)$, $(45, 158)$, $(46, 152)$, $(47, 152)$, and $(48, 150)$. The pair $(47, 152)$ is not a Pareto optimum; it is dominated by $(46, 152)$. The drawings in Fig. 6 were generated using the same scripts as those in Fig. 5. An argument can be made that the two intermediate drawings, (b) and (c), are more aesthetically pleasing than the extremes.



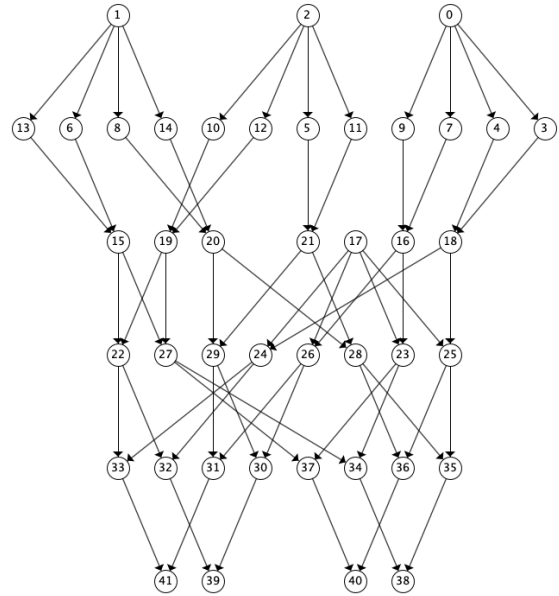
(a) 44 crossings, non-verticality= 172



(b) 45 crossings, non-verticality= 158



(c) 46 crossings, non-verticality= 152



(d) 48 crossings, non-verticality= 150

Figure 6: Four drawings of `north42.32.GKNV`, showing all possible tradeoffs between total crossings and non-verticality.

3 Heuristics

Here we describe software that implements a large collection of simple heuristics targeting the various layered graph objectives.

3.1 Iterative improvement

Iterative improvement heuristics start with a given configuration, random or the result of preprocessing – see Section 3.4, and perform rearrangements until a stopping criterion is achieved. Each heuristic defines an *iteration*, a single rearrangement of nodes on a layer, and a *pass*, a sequence of iterations that either systematically rearrange every layer or every single node. Heuristics terminate either after a fixed number of passes or when a pass fails to improve the objective or any of a set of objectives.

3.1.1 Basic paradigms

There are two basic types of iterative improvement heuristics: sorting and node-insertion.

Sorting. Each iteration of a sorting heuristic works as follows.

1. choose a layer k
2. assign weights to nodes on layer k based on positions of their neighbors on layers $k - 1$ and/or $k + 1$
3. sort the nodes on layer k by increasing weight

Examples of sorting heuristics are the median and the barycenter heuristics – see [3]. Let L be the number of layers. Then in both of these heuristics a pass consists of an *upsweep*, where $k = 2, \dots, L$ and weights are based on layer $k - 1$, followed by a *downsweep*, where $k = L - 1, \dots, 1$ and weights are based on layer $k + 1$. The difference is that in the median heuristic, the weight of x is the median of the positions of its neighbors, whereas in the barycenter it is the mean.

In a variant called *mod-bary*, the survivor among several variants of *maximum crossings layer* [?], each pass works as follows.

1. choose unmarked layer k^* whose incident edges have the maximum number of crossings and mark it
2. sort nodes on layer k^* with weights based on neighbor positions on layers $k^* - 1$ (if $k^* > 1$) and $k^* + 1$ (if $k^* < L$)
3. do an upsweep for layers $k^* + 1, \dots, L - 1$
4. do a downsweep for layers $k^* - 1, \dots, 1$
5. the pass ends when all layers are marked

In many cases *mod-bary* outperforms barycenter (which, in turn, almost always outperforms the median heuristic).

Node-insertion. Each pass of a node-insertion heuristic works as follows.

1. choose unmarked node x
2. put x into an optimal position on its layer based on some objective, shifting other nodes as needed

The first published node-insertion heuristic, called *sifting*, is due to Matuszewski et al. [7]. In their version no marking takes place. Nodes are simply considered by increasing or decreasing degree in each pass. Each x in turn is moved to a position that minimizes total crossings. The heuristic switches to the opposite order when a pass does not yield improvement.

Another node-insertion heuristic, specifically designed for the bottleneck crossing problem, is *maximum crossings edge (mce)* [9]. To choose node x , the mce heuristic first identifies the edge vw with the maximum number of crossings, making sure that at least one of v, w is unmarked. Then v and/or w is moved. Suppose $x = v$ is moved. Instead of moving x to a position that minimizes the total number of crossings overall, as sifting does, mce moves x to a position that minimizes the maximum number of crossings for any edge incident on x .

A simpler version of mce, which targets total crossings, is *maximum crossings node (mcn)* (see Gupta [?]). Here x is the node whose incident edges have the most crossings and, as in sifting, x is moved to the position that minimizes total crossings.

The mce and mcn heuristics motivate two other variations: mce-t, where x is chosen as in mce but the position of x is chosen to minimize total crossings; and mcn-e, where x is the (unmarked) node with the maximum number of crossings but the position of x is based on minimizing the maximum number of crossings for any edge incident on x .

3.2 Verticality

Optimizing verticality, i.e., minimizing non-verticality, requires special care. When n_ℓ , the number of nodes on layer ℓ is less than W , the largest number of nodes on any layer, the positions of nodes on layer ℓ may not be contiguous. This means that the sorted order of nodes on ℓ does not uniquely determine node positions and placing a node in its optimum position on layer ℓ may not involve shifting of other nodes.

3.2.1 Vertical barycenter

We implemented a non-verticality-minimizing heuristic based on mod-bary, called *vertical-bary*. Each iteration starts the same way as in mod-bary, with k^* the layer whose incident edges have maximum total nonverticality. Then, to choose positions based on the sorted order, we can apply a dynamic programming algorithm that chooses optimum positions for layer ℓ given fixed positions on the relevant neighboring layers and the sorted order of nodes on the current layer ℓ . The relevant neighboring layers are $\ell - 1$ and $\ell + 1$ if $\ell = k^*$ and either $\ell - 1$ or $\ell + 1$ otherwise.

Let $d_{x,p}$ be the nonverticality induced by putting node x in position p on layer ℓ and let $p^*(x)$ be the position of x in an assignment of positions that minimizes $\sum_{x \text{ on } \ell} d_{x,p^*(x)}$. Our goal is to find $p^*(x)$ for all x on ℓ .

Let $C[i, j]$ = the minimum total nonverticality for nodes $1, \dots, i$ (in the sorted order) using positions $1, \dots, j$ only, and let $P[i, j] = \mathbf{true}$ if the optimum position for i is j , **false** otherwise. Starting with $C[0, 0] = 0$ and $C[j, 0] = \infty$ for all j as base cases, $C[i, j]$ is the minimum of

$$\begin{array}{ll} C[i-1, j-1] + d_{i,j} & \text{put node } i \text{ in position } j \\ C[i, j-1] & \text{put node } i \text{ in a position } < j \end{array}$$

If the first option is minimum, then $P[i, j] = \mathbf{true}$, otherwise $P[i, j] = \mathbf{false}$.

Note that the desired total nonverticality is $C[n_\ell, W]$ and we only need $i \leq j \leq W$ and $i \leq n_\ell$. The number of table entries in a dynamic programming algorithm, ignoring the base cases, is therefore $n_\ell(W - n_\ell)$, the worst case being when $n_{ell} \approx W/2$.

To extract $p^*(i)$ for $i = 1, \dots, n_\ell$ apply the following recursive procedure starting with $\text{Positions}(n_\ell, W)$.

```
procedure Positions( $i, j$ )
  if  $i = 0$  return
  if  $P[i, j]$ 
```

```

     $p^*(i) = j$ 
    Positions( $i - 1, j - 1$ )
else
    Positions( $i, j - 1$ )
end

```

3.2.2 Maximum nonverticality edge

The *maximum nonverticality edge* (*mnve*) heuristic behaves in a way similar to mce with the following observations.

- It is more like mce-n in that it positions node x to minimize total nonverticality.
- The optimum position for x could be unoccupied: in that case no shifting takes place.

A variant that minimizes the maximum nonverticality among edges incident on x with target objective bottleneck nonverticality could be easily implemented.

3.3 Combining heuristics

3.4 Preprocessing

Experiments have shown that preprocessing the graph before applying an iterative improvement heuristic leads to better and more consistent results. The most effective techniques involve depth-first or breadth-first search followed with a sort of each layer by increasing discovery time of the nodes. For two-layer instances breadth-first search, particularly a guided two-pass version, is especially effective – see [8]. With more layers depth-first search is a better choice.

The sorting based on these graph traversals brings nodes that are close to each other in the graph near each other on their respective layers.

3.5 Post processing

[for large scale experimental results using mod-bary with iterative improvement heuristics, go to the research drive rs1 and look under Old-Research/Layered-Graphs-Old/LargeScale-Experiments]

4 Random Generation

5 Conclusions

Acknowledgements

Several students have contributed to this work either as developers and experimenters. Saurabh Gupta, in research leading to his 2008 master’s thesis, implemented the first version of what is now **minimization**. It incorporated a large variety of heuristics including, among others: barycenter, median, mod-bary, sifting, mcn, and dfs.

In later independent study projects, Venkata Uday K. Sonthy (2014) implemented programs and scripts to convert layered graphs to ILP’s for crossing minimization and extracting solutions; Weijia Li (2016) explored tradeoffs between stretch and crossings (including their bottleneck versions), adding necessary heuristics; Mason Hicks (2022) added implementations of heuristics for minimizing nonverticality. All of these students undertook large scale experiments.

We also acknowledge the computing resources provided by North Carolina State University High Performance Computing Services Core Facility (RRID:SCR_022168).

References

- [1] Markus Chimani and Philipp Hungerländer. Exact approaches to multi-level vertical orderings. *Inform. Journal on Computing*, pages 611–624, 2013.
- [2] CPLEX. IBM ILOG CPLEX Optimization Studio. <http://www-01.ibm.com/software/-commerce/optimization/cplex-optimizer/>.
- [3] G. Di Battista, P. Eades, R. Tamassia, and I. G. Tollis. *Graph Drawing: Algorithms for the Visualization of Graphs*. Prentice Hall, 1999.
- [4] Galant: A **G**raph **A**lgorithm **A**nimation **T**ool. <https://github.com/mfms-ncsu/galant>.
- [5] E. R. Gansner, E. Koutsifios, S. C. North, and K. P. Vo. A technique for drawing directed graphs. *IEEE Transactions on Software Engineering*, 19:214–230, 1993.
- [6] M. Jünger and P. Mutzel. 2-Layer Straightline Crossing Minimization: Performance of Exact and Heuristic Algorithms. *Journal of Graph Algorithms and Applications (JGAA)*, 1(1):1–25, 1997.
- [7] C. Matuszewski, R. Schöfeld, and P. Molitor. Using sifting for k -layer straightline crossing minimization. In *Graph Drawing*, number 1731 in Lecture Notes in Computer Science, pages 217–224, 1999.
- [8] M. Stallmann, F. Brglez, and D. Ghosh. Heuristics, Experimental Subjects, and Treatment Evaluation in Bigraph Crossing Minimization. *Journal on Experimental Algorithmics*, 6(8), 2001.
- [9] Matthias Stallmann. A heuristic for bottleneck crossing minimization and its performance on general crossing minimization: Hypothesis and experimental study. *Journal on Experimental Algorithmics*, 17, 2012.
- [10] Matthias Stallmann. Edge offset in drawings of layered graphs with evenly-spaced nodes on each layer. Technical Report TR-2016-6, Computer Science Department, North Carolina State University, 2016.